

75.56220152PEC2Solucion.pdf



Jose_Blasco_83



Fundamentos de Computadores



1º Grado en Ingeniería Informática



**Facultad de Informática, Multimedia y Telecomunicación
Universitat Oberta de Catalunya**



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



PEC2 - Segunda prueba de evaluación continuada

Presentación

Después de conocer los sistemas de representación de la información, esta PEC se focaliza en los circuitos lógicos combinacionales. Las funciones lógicas nos permiten describir la funcionalidad de un circuito y mediante las puertas lógicas y los bloques combinacionales podemos implementarlas en un circuito. Todo esto, no se podría hacer sin conocer mecanismos de minimización, como mapas de Karnaugh, que nos permiten reducir las dimensiones del circuito a implementar.

Competencias

- Conocer y saber aplicar el álgebra de Boole para la manipulación de funciones lógicas.
- Tener nociones tecnológicas de los circuitos digitales y entender la relación entre los circuitos digitales y las funciones lógicas.
- Conocer y saber utilizar las puertas lógicas y los módulos combinacionales en el diseño de circuitos lógicos.

Objetivos

- Saber aplicar las operaciones básicas y los axiomas del algebra de Boole.
- Saber representar las funciones lógicas mediante tablas de verdad.
- Saber representar las funciones lógicas mediante expresiones algebraicas.
- Saber analizar un circuito combinacionales.
- Saber realizar un cronograma a partir de un circuito digital combinacional.
- Saber sintetizar una función a dos niveles.
- Saber diseñar un circuito combinacional sencillo a partir de los bloques combinacionales de los materiales.

Recursos

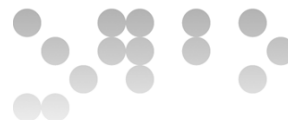
Los recursos que se recomienda usar por esta PEC son los siguientes:

Básicos: El módulo 3 de los materiales.

Complementarios: KeMap, VerilCIRC, VerilCHART y el Wiki de la asignatura. También tenéis disponible el servicio PyPAC para obtener beneficios en la asignatura por la realización de ejercicios en VerilUOC.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración esta indicada en cada uno de los subapartados.



Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado debéis dirigiros al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF utilizando una de las plantillas entregadas conjuntamente con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **6 de abril** (a las 24 horas).

Solución de la PEC

Ejercicio 1 [15%]

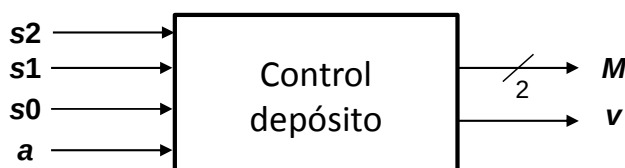
En un depósito con una bomba se mide continuamente el nivel del agua con tres sensores. Cada sensor dispone de un indicador binario. Con los indicadores se detectan diferentes niveles de agua en el depósito.

Entradas del sistema:

- Tres sensores de agua con las señales s_2 , s_1 y s_0 . El sensor s_2 se activa a 1 cuando el nivel de agua está por debajo del **90%** de la capacidad del depósito, se pone a 0 de lo contrario. El sensor s_1 se activa a 1 cuando el nivel de agua está por debajo del **20%** de la capacidad del depósito, se pone a 0 de lo contrario. El sensor s_0 se activa a 1 cuando el nivel de agua está por debajo del **5%** de la capacidad del depósito, se pone a 0 de lo contrario.
- Una señal a de un bit que tiene el valor 1 cuando la bomba está en marcha.

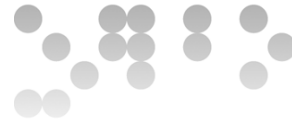
Salidas del sistema:

- Una señal M de 2 bits para controlar un motor.
- Una señal v de 1 bit para abrir una válvula.



Funcionamiento del sistema:

- La señal M tiene que codificar 11 cuando el nivel de agua es igual o superior al 90%.
- La señal M tiene que codificar 10 cuando el nivel de agua está por debajo del 90% e igual o superior al 20%.
- La señal M tiene que codificar 01 cuando el nivel de agua está por debajo del 20% e igual o superior al 5%.



- d) Cuando el nivel de agua está por debajo del 5%, la señal M tiene que codificar 00.
- e) La señal v de la válvula tiene que activarse a 1 únicamente cuando el nivel de agua esté por debajo del 5% y la bomba esté en marcha.

Obtened la tabla de verdad de M y v .

NOTA: Tened en cuenta los casos no importa (*don't care*) que se dan en la entrada.

s_2	s_1	s_0	a	m_1	m_0	v
0	0	0	0			
0	0	0	1			
0	0	1	0			
0	0	1	1			
0	1	0	0			
0	1	0	1			
0	1	1	0			
0	1	1	1			
1	0	0	0			
1	0	0	1			
1	0	1	0			
1	0	1	1			
1	1	0	0			
1	1	0	1			
1	1	1	0			
1	1	1	1			

La tabla de verdad tiene que indicar cuál es el valor de las salidas M de dos bits (m_1 , m_0) y de v para todas las combinaciones de las 4 variables de entrada: s_2 , s_1 , s_0 , a .

A partir de la descripción del funcionamiento de las variables de entrada observamos que hay combinaciones que no se pueden dar nunca. Por ejemplo, nunca puede haber un sensor de un nivel de agua más alto con valor a 0 (nivel de agua por encima del sensor) y otro sensor de un nivel de agua más bajo con valor 1 (nivel de agua por debajo del sensor). Estas combinaciones que no se pueden producir (no importa o *don't care*) tienen el valor X en la salida.

Completamos la tabla de verdad de la salida M y v en función de los casos descritos en el enunciado:

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

75.562 · Fundamentos de Computadors · PEC2 2015–2016 · Estudios de Informática Multimèdia y Telecomunicación



perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

s2	s1	s0	a	m1	m0	v
0	0	0	0	1	1	0
0	0	0	1	1	1	0
0	0	1	0	X	X	X
0	0	1	1	X	X	X
0	1	0	0	X	X	X
0	1	0	1	X	X	X
0	1	1	0	X	X	X
0	1	1	1	X	X	X
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	X	X	X
1	0	1	1	X	X	X
1	1	0	0	0	1	0
1	1	0	1	0	1	0
1	1	1	0	0	0	0
1	1	1	1	0	0	1

Ejercicio 2 [25%]

Considerad la siguiente tabla de verdad

a	b	c	d	f
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	X
1	0	1	1	X
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Se pide que:

- a) [15%] Sintetizéis e implementéis con puertas lógicas de manera mínima a dos niveles la función f mediante el método de Karnaugh.

NOTA: Tenéis disponible el ejercicio en VerilCIRC y en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que



se hagan más de 5 intentos, la nota del ejercicio se reducirá al 50%. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC. En el caso de VerilCIRC para los circuitos, no hay limitación en el número de intentos.

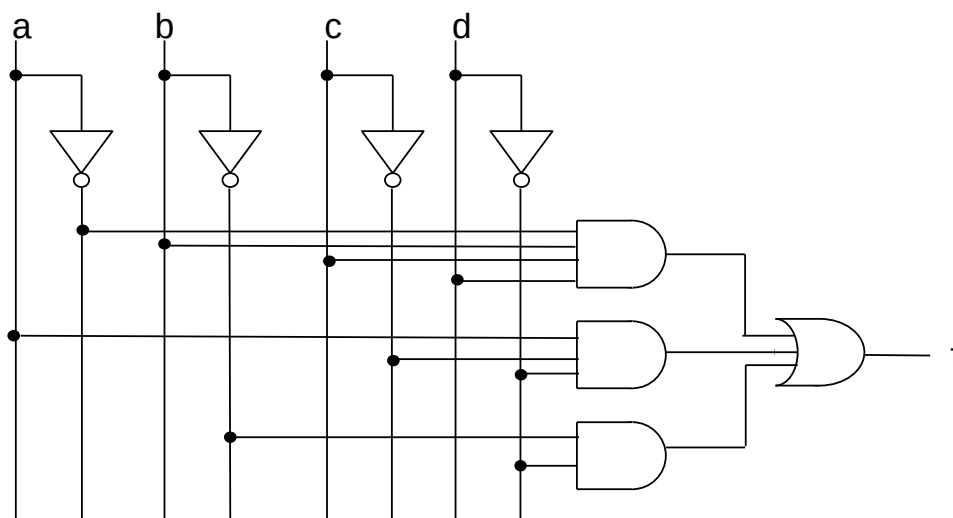
Pasamos los valores de la tabla de verdad al mapa de Karnaugh y agrupamos los unos adyacentes. El mapa de Karnaugh para la función de salida f es el siguiente:

$a \backslash b$ $c \ d$	00	01	11	10
00	1	0	1	1
01	0	0	0	0
11	0	1	0	x
10	1	0	0	x

$$f = a'bcd + ac'd' + b'd'$$

Donde " ' ", que también se puede representar como " ~ ", representa la negación de la variable de entrada.

Dibujamos el circuito correspondiente a la salida f :

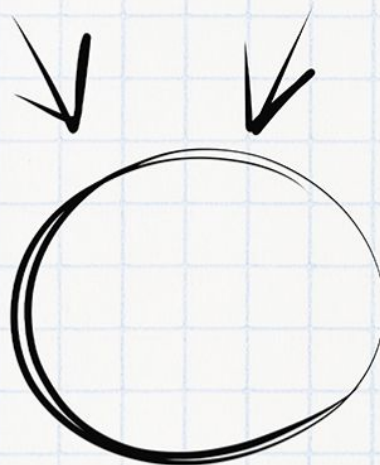


Imagínate aprobando el examen

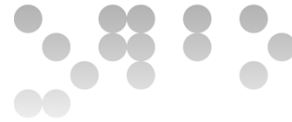
Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



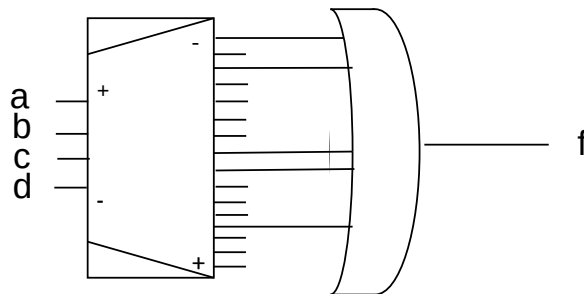
WUOLAH



- b) [10%] Implementéis la función f utilizando un decodificador de 4 entradas y las puertas lógicas que consideréis necesarias.

NOTA: Tenéis disponible el ejercicio en VerilCIRC. En este caso no hay limitación en el número de intentos.

Las entradas a b c d serán las entradas del decodificador. Para obtener la tabla de verdad de f , juntamos mediante una puerta OR estas salidas del decodificador donde según la tabla de verdad la función $f = 1$. En este caso, las entradas con *don't care* se pueden añadir o no a la puerta OR. En la implementación, no se han añadido para reducir la dimensión de la puerta OR. La implementación resultante es la siguiente:

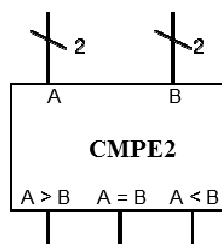


Ejercicio 3 [35%]

Se quiere diseñar un circuito combinacional que permita hacer la comparación de números enteros representados en complemento a 2.

El diseño lo haremos en dos partes:

- a) [10%] Considerad que las entradas A y B son números enteros representados en complemento a 2 y de 2 bits cada entrada. El circuito llamado CMPE2 sería el siguiente:



Se pide que rellenéis la tabla de verdad siguiente con las salidas correspondientes a este comparador.

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

75.562 · Fundamentos de Computadors · PEC2 2015–2016 · Estudios de Informática Multimèdia y Telecomunicación



perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

a1	a0	b1	b0	A>B	A=B	A<B
0	0	0	0			
0	0	0	1			
0	0	1	0			
0	0	1	1			
0	1	0	0			
0	1	0	1			
0	1	1	0			
0	1	1	1			
1	0	0	0			
1	0	0	1			
1	0	1	0			
1	0	1	1			
1	1	0	0			
1	1	0	1			
1	1	1	0			
1	1	1	1			

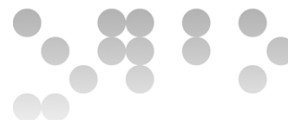
NOTA: Tenéis disponible el ejercicio en VerilCHART, donde las señales de salida están representadas por *gt* ($A > B$), *eq* ($A = B$) y *lt* ($A < B$) respectivamente.

Un número en dos bits y Ca2 corresponde a los siguientes números enteros en decimal:

b1	b0	Número entero decimal
0	0	0
0	1	1
1	0	-2
1	1	-1

Rellenamos la tabla de verdad para los casos $A > B$, $A = B$ y $A < B$:

a1	a0	b1	b0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	1	0	0
1	1	1	1	0	1	0



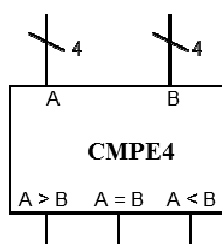
- b) [5%] Implementad el circuito CMPE2 definido en el apartado anterior con una memoria ROM. Utilizad una memoria ROM con el mínimo número de señales de direccionamiento y de datos que haga falta.

NOTA: Tenéis disponible el ejercicio en VerilCIRC, donde las señales de salida están representadas por *gt* ($A > B$), *eq* ($A = B$) y *lt* ($A < B$) respectivamente. En este caso no hay limitación en el número de intentos.

En la memoria ROM que implemente la tabla de verdad, la señal de direccionamiento @ es de 4 bits y corresponde a los 4 bits de entrada, $@ = a1\ a0\ b1\ b0$, y el contenido de la ROM $M[@]$ corresponde a los valores de la tabla de verdad para $A > B$, $A = B$ y $A < B$. La salida de la ROM pues es de 3 bits.

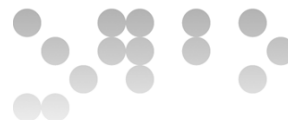
@	M[@]		
0	0	1	0
1	0	0	1
2	1	0	0
3	1	0	0
4	1	0	0
5	0	1	0
6	1	0	0
7	1	0	0
8	0	0	1
9	0	0	1
10	0	1	0
11	0	0	1
12	0	0	1
13	0	0	1
14	1	0	0
15	0	1	0

Se trata ahora de diseñar un circuito CMPE4 donde tenemos como entradas números enteros representados en Ca2 de 4 bits.



Para simplificar la resolución del ejercicio sólo tendréis que realizar el circuito de dos salidas y lo haremos por separado.

- c) [15%] Diseñad la salida $A > B$ del circuito CMPE4. Podéis usar cualquier de los bloques combinacionales vistos en el material, incluso el bloque hecho en el apartado b). También se pueden usar las puertas lógicas que consideréis



necesarias. No se permite solucionar el problema usando memorias ROM, excepto la diseñada en el apartado b).

NOTA: Tenéis disponible el ejercicio en VerilCIRC donde la señal de salida $A > B$ está representado por *gt*. En el supuesto de que reutilizéis el circuito del apartado b), tendréis que copiar la ROM con todos sus valores.. En este caso no hay limitación en el número de intentos.

El caso de $A > B$, considerando el signo de A y B indicado por los bits a_3 y b_3 , se puede producir en las condiciones que indica la tabla siguiente:

A	B	$A > B$
+	+	para determinar
+	-	siempre
-	+	nunca
-	-	para determinar

- Caso (+ +): Cuando los dos números son positivos, a_3 es 0 y b_3 es 0. Comprobaremos este caso con una puerta AND con entradas a_3' y b_3' , y un comparador. La salida $A > B$ del comparador indica si el valor de A es mayor que B . Por ejemplo: $A = +7_{(10)} = 0111_{(Ca2)}$ y $B = +5_{(10)} = 0101_{(Ca2)}$. La salida $A > B$ del comparador se activaría a 1.
- Caso (- -): Cuando los dos números son negativos, a_3 es 1 y b_3 es 1. Comprobaremos este caso con una puerta AND con entradas a_3 y b_3 , y un comparador. La salida $A > B$ del comparador indica si el valor de A es mayor que B . Notad que en Complemento a 2 los números negativos mantienen el orden siendo $1111_{(Ca2)}$ ($-1_{(10)}$) el valor más grande y $1000_{(Ca2)}$ ($-8_{(10)}$) el valor más pequeño. Por ejemplo $A = -7_{(10)} = 1001_{(Ca2)}$ y $B = -5_{(10)} = 1011_{(Ca2)}$. La salida $A > B$ del comparador se activaría a 1.
- Caso (+ -): En este caso, A es positivo y B es negativo. A siempre es mayor que B . Comprobaremos esta combinación de signos con una puerta AND y los bits a_3' y b_3 .

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

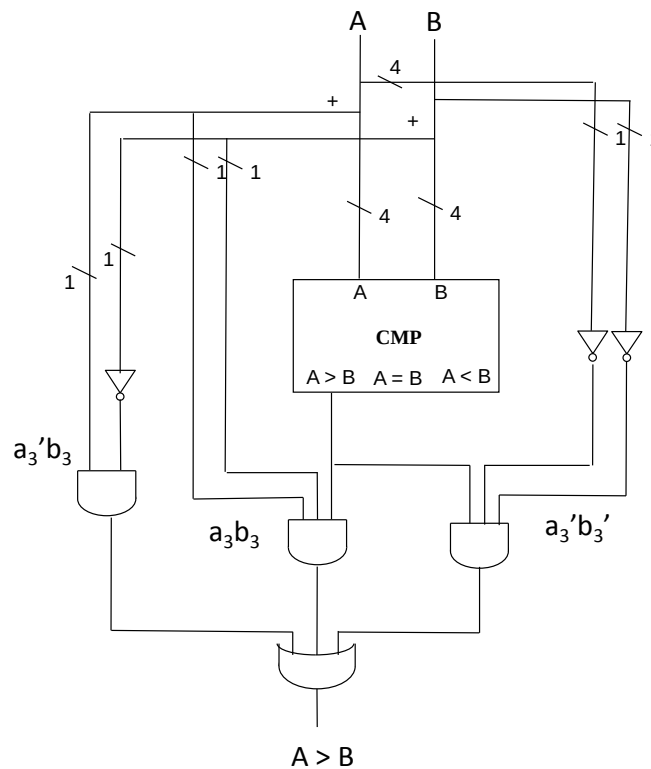
pierdo espacio



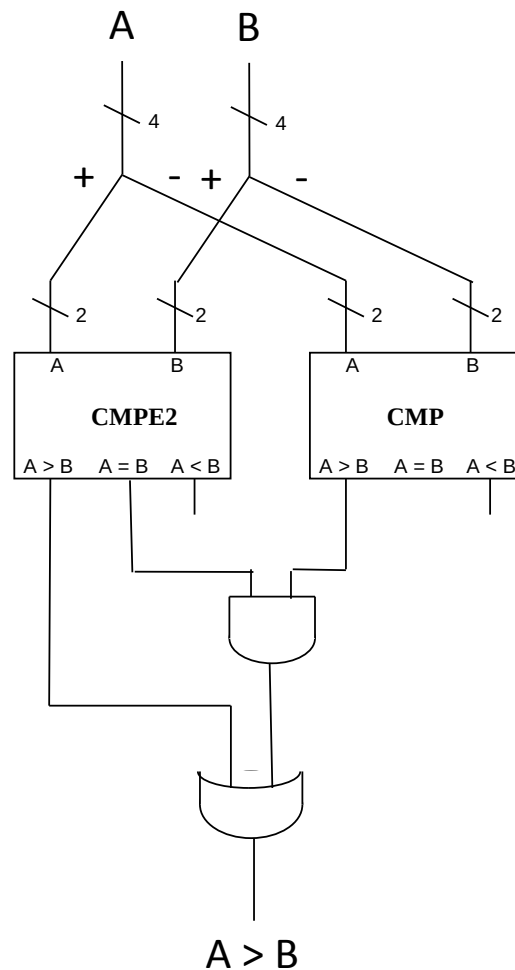
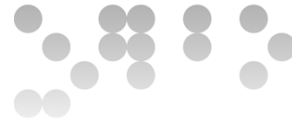
Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH



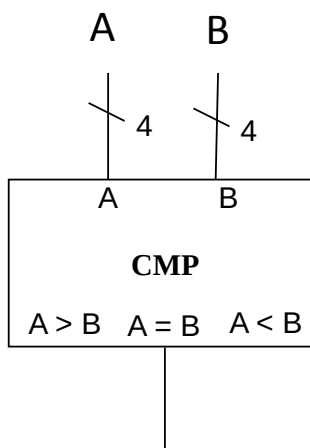
El bloque CMPE4 también se puede implementar utilizando el bloque CMPE2 diseñado en el apartado b) y un bloque CMP. El bloque CMPE2 comprueba $A > B$ en los dos bits de más peso. La salida $A > B$ indica todos los casos de A mayor que B basados en los dos bits de más peso, pero no puede detectar los casos cuando la información se encuentra en los bits de menos peso. Por eso, cuando $a_3a_2 = b_3b_2$, que significa que A y B tienen el mismo signo (+ + o - -) y el mismo valor en los bits a_2 y b_2 , se necesita un bloque comparador de 2 bits para comparar los bits de menor peso a_1a_0 y b_1b_0 . La salida $A > B$ del comparador indica si finalmente A es mayor que B.



- d) [5%] Diseñad la salida $A=B$ del circuito CMPE4. Podéis usar cualquier de los bloques combinacionales vistos en el material, incluso el bloque hecho en el apartado b). También se pueden usar las puertas lógicas que consideréis necesarias. No se permite solucionar el problema usando memorias ROM.

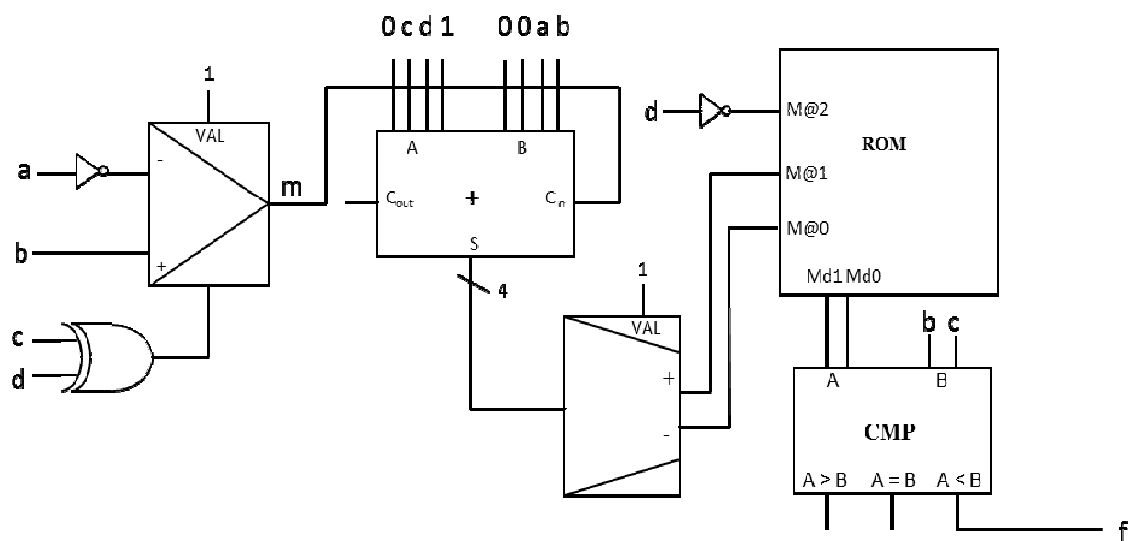
NOTA: Tenéis disponible el ejercicio en VerilCIRC donde la señal de salida $A = B$ está representado por *eq*. En el supuesto de que reutilizéis el circuito del apartado b), tendréis que copiar la ROM con todos sus valores. En este caso no hay limitación en el número de intentos.

La condición $A=B$ se cumple cuando los valores de los 4 bits de A y B coinciden. Podemos usar un bloque CMP de 4 bits. Usamos la salida $A=B$ del comparador para detectar este caso.



Ejercicio 4 [25%]

Dado el circuito lógico combinacional siguiente:



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

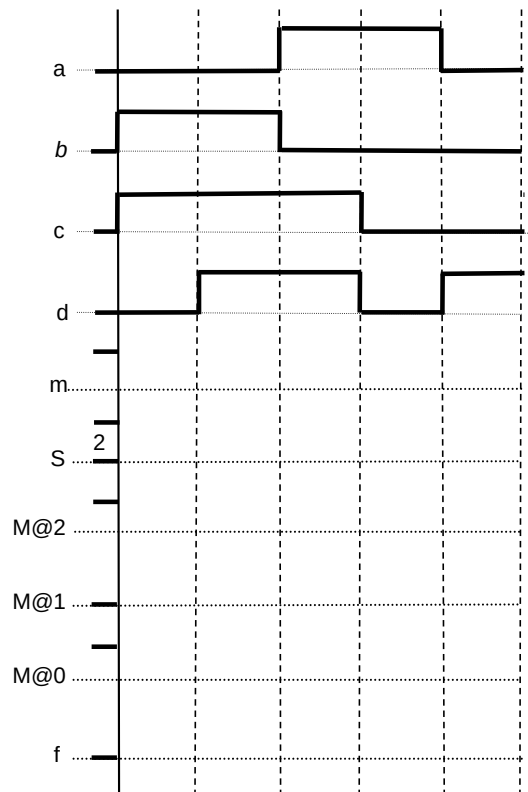
75.562 · Fundamentos de Computadors · PEC2 2015–2016 · Estudios de Informática Multimèdia y Telecomunicación

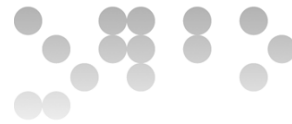


Donde la memoria ROM tiene el contenido siguiente:

@	M[@]
0	10
1	00
2	01
3	11
4	11
5	00
6	11
7	10

Completad el cronograma siguiente que muestra el valor de la salida f así como el valor de otras señales intermedias: m (salida del multiplexor), S (salida de 4 bits del sumador) y $M@$ (entrada de direccionamiento de 3 bits de la memoria ROM). La señal S se tiene que mostrar como un bus con su valor representado en hexadecimal. En cambio, la señal $M@$ se tiene que mostrar con cada bit por separado.





NOTA: Tenéis disponible el ejercicio en VerilCHART.

Para obtener con más facilidad las salidas del circuito usamos la siguiente tabla para los valores de señales internas del circuito.

					A	B	S						
a	b	c	d	m	0 c d 1	0 0 a b	S ₃ S ₂ S ₁ S ₀	M @ ₂	M @ ₁	M @ ₀	M d ₁	M d ₀	f
0	1	1	0	1	0 1 0 1	0 0 0 1	0 1 1 1	1	1	0	1	1	0
0	1	1	1	1	0 1 1 1	0 0 0 1	1 0 0 1	0	1	1	1	1	0
1	0	1	1	0	0 1 1 1	0 0 1 0	1 0 0 1	0	1	1	1	1	0
1	0	0	0	0	0 0 0 1	0 0 1 0	0 0 1 1	1	0	1	0	0	0
0	0	0	1	0	0 0 1 1	0 0 0 0	0 0 1 1	0	0	1	0	0	0

Representamos los valores obtenidos en el cronograma.

